

**EX. NO:1****ARRAYS, STRUCTURES AND POINTERS****DATE:****QUESTION 1:**

1(a) Implement Insertion (beginning, middle, and end), deletion (beginning, middle, end), searching and Display operations on arrays.

**AIM:**

To write a C program to implement insertion (beginning, middle, and end), deletion (beginning, middle, end), searching and Display operations on arrays.

**ALGORITHM:**

Step-1: start

Step-2: define the following functions.

1. Function `insert_beg(a[], e, s)`:
  - Accepts parameters: `a[]` (the array), `e` (element to insert at the beginning), `s` (current size).
  - Loop from `i = s` down to 1:
  - Shift elements to the right to make space for the new element.
  - Insert `e` at the beginning (at `a[0]`).
  - Increment the size `s`.
2. Function `insert_end(a[], e, s)`:
  - Accepts parameters: `a[]` (the array), `e` (element to insert at the end), `s` (current size).
  - Insert `e` at the end (at `a[s]`).
  - Increment the size `s`.
3. Function `insert_pos(a[], e, p, s)`:
  - Accepts parameters: `a[]` (the array), `e` (element to insert), `p` (position), `s` (current size).
  - Loop from `i = s` down to `p - 1`:
  - Shift elements to the right to make space for the new element.
  - Insert `e` at position `p-1`.
  - Increment the size `s`.
4. Function `del_beg(a[], s)`:
  - Accepts parameters: `a[]` (the array), `s` (current size).
  - Shift elements to the left to remove the first element.
  - Decrement the size `s`.

5. Function `del\_pos(a[], p, s)`:

- Accepts parameters: `a[]` (the array), `p` (position to delete), `s` (current size).
- Loop from `i = p - 1` up to `s`:
- Shift elements to the left to remove the element at position `p-1`.
- Decrement the size `s`.

6. Function `search(a[], e, s)`:

- Accepts parameters: `a[]` (the array), `e` (element to search), `s` (current size).
- Iterate through the array to check if `e` is present.
- Display the position if found or a message if not found.

Step-3: in main function using switch case and do while loop get the user's choice.

Step-4: stop.

### PROGRAM:

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void display(int arr[], int n)
```

```
{
    int i;
    if(n == 0)
    {
        printf("Array is empty.\n");
        return;
    }

```

```
    printf("Array Elements: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
void insertBeginning(int arr[], int *n, int value)
```

```
{
    int i;
    for(i = *n; i > 0; i--)
        arr[i] = arr[i-1];

```

```
    arr[0] = value;
    (*n)++;
}
```

```
void insertEnd(int arr[], int *n, int value)
```

```
{
    arr[*n] = value;
    (*n)++;
}
```

```
void insertMiddle(int arr[], int *n, int pos, int value)
{
    int i;

    if(pos < 1 || pos > *n + 1)
    {
        printf("Invalid Position!\n");
        return;
    }

    for(i = *n; i >= pos; i--)
        arr[i] = arr[i-1];

    arr[pos-1] = value;
    (*n)++;
}

void deleteBeginning(int arr[], int *n)
{
    int i;

    if(*n == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    for(i = 0; i < *n-1; i++)
        arr[i] = arr[i+1];

    (*n)--;
}

void deleteEnd(int *n)
{
    if(*n == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    (*n)--;
}

void deleteMiddle(int arr[], int *n, int pos)
{
    int i;

    if(pos < 1 || pos > *n)
    {
        printf("Invalid Position!\n");
        return;
    }
}
```

```
for(i = pos-1; i < *n-1; i++)
arr[i] = arr[i+1];

(*n)--;
}

void search(int arr[], int n, int key)
{
int i;

for(i = 0; i < n; i++)
{
if(arr[i] == key)
{
printf("Element found at position %d\n", i+1);
return;
}
}

printf("Element not found.\n");
}

void mergeArrays(int arr1[], int n1, int arr2[], int n2)
{
int merge[MAX];
int i;

for(i = 0; i < n1; i++)
merge[i] = arr1[i];

for(i = 0; i < n2; i++)
merge[n1+i] = arr2[i];

printf("Merged Array: ");
for(i = 0; i < n1+n2; i++)
printf("%d ", merge[i]);

printf("\n");
}

void checkSorted(int arr[], int n)
{
int i;

for(i = 0; i < n-1; i++)
{
if(arr[i] > arr[i+1])
{
printf("Array is NOT Sorted.\n");
return;
}
}
}
```

```
printf("Array is Sorted.\n");
}

int main()
{
int arr[MAX], arr2[MAX];
int n, n2;
int choice, value, pos, key;
int i;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements:\n");
for(i = 0; i < n; i++)
scanf("%d", &arr[i]);

do
{
printf("\n----- MENU ----- \n");
printf("1. Insert at Beginning\n");
printf("2. Insert at Middle\n");
printf("3. Insert at End\n");
printf("4. Delete at Beginning\n");
printf("5. Delete at Middle\n");
printf("6. Delete at End\n");
printf("7. Search\n");
printf("8. Merge Arrays\n");
printf("9. Check Sorted\n");
printf("10. Display\n");
printf("11. Exit\n");
printf("Enter Choice: ");
scanf("%d", &choice);

switch(choice)
{
case 1:
printf("Enter value: ");
scanf("%d", &value);
insertBeginning(arr, &n, value);
break;

case 2:
printf("Enter position: ");
scanf("%d", &pos);
printf("Enter value: ");
scanf("%d", &value);
insertMiddle(arr, &n, pos, value);
break;

case 3:
printf("Enter value: ");
scanf("%d", &value);
insertEnd(arr, &n, value);
```

```
break;

case 4:
deleteBeginning(arr, &n);
break;

case 5:
printf("Enter position: ");
scanf("%d", &pos);
deleteMiddle(arr, &n, pos);
break;

case 6:
deleteEnd(&n);
break;

case 7:
printf("Enter element to search: ");
scanf("%d", &key);
search(arr, n, key);
break;

case 8:
printf("Enter number of elements in second array: ");
scanf("%d", &n2);

printf("Enter second array elements:\n");
for(i = 0; i < n2; i++)
scanf("%d", &arr2[i]);

mergeArrays(arr, n, arr2, n2);
break;

case 9:
checkSorted(arr, n);
break;

case 10:
display(arr, n);
break;

case 11:
printf("Program Ended.\n");
break;

default:
printf("Invalid Choice!\n");
}

} while(choice != 11);

return 0;
}
```

**OUTPUT:**

```

[ita2536@writlin harshan]$ cc ex1.1.c
[ita2536@writlin harshan]$ vi ex1.1.c
[ita2536@writlin harshan]$ ./a.out
Enter number of elements: 4
Enter elements:
2
4
6
8

----- MENU -----
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete at Beginning
5. Delete at Middle
6. Delete at End
7. Search
8. Merge Arrays
9. Check Sorted
10. Display
11. Exit
Enter Choice: 2
Enter position: 2
Enter value: 10

----- MENU -----
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete at Beginning
5. Delete at Middle
6. Delete at End
7. Search
8. Merge Arrays
9. Check Sorted
10. Display
11. Exit
Enter Choice: 10
Array Elements: 2 10 4 6 8

----- MENU -----
1. Insert at Beginning
2. Insert at Middle
3. Insert at End
4. Delete at Beginning
5. Delete at Middle
6. Delete at End
7. Search
8. Merge Arrays
9. Check Sorted
10. Display
11. Exit
Enter Choice: 11
Program Ended.
[ita2536@writlin harshan]$

```

**RESULT:**

Thus, the C program to implement insertion (beginning, middle, and end), deletion (beginning, middle, end), searching and Display operations on arrays was executed successfully.

**QUESTION 2:**

1(b) To implement Pointers to structures for the below scenario.

Read the details of 2 employees (Name, EMP\_ID, Salary) using pointers to structures.

Display the details of the employee with maximum salary using a function

high\_paid\_employee(..) by applying the concept of pass by address. Use functions to read and display employee details.

**AIM:**

To implement pointers to structures in C by reading the details of two employees (Name, EMP\_ID, and Salary) using pointers, and to display the details of the employee with the maximum salary using the high\_paid\_employee() function with the concept of pass by address.

**ALGORITHM:**

1. Start the program.
2. Define a structure Employee with members: Name, EMP\_ID, and Salary.
3. Create two structure variables for storing employee details.
4. Read the details of both employees using the readEmployee() function and pointers to structures.
5. Compare the salaries of the two employees using the high\_paid\_employee() function (pass by address).
6. Display the details of the employee with the maximum salary using the displayEmployee() function.
7. Stop the program.

**PROGRAM:**

```
#include <stdio.h>
struct Employee
{
    char name[30];
    int empid;
    float salary;
};

void read(struct Employee *e)
{
    printf("Enter Name: ");
    scanf("%s", e->name);

    printf("Enter Employee ID: ");
    scanf("%d", &e->empid);

    printf("Enter Salary: ");
    scanf("%f", &e->salary);
}
```

```
void display(struct Employee *e)
{
    printf("\nEmployee Details\n");
    printf("Name      : %s\n", e->name);
    printf("Employee ID : %d\n", e->empid);
    printf("Salary   : %.2f\n", e->salary);
}

void high_paid_employee(struct Employee *e1, struct Employee *e2)
{
    if(e1->salary > e2->salary)
        display(e1);
    else
        display(e2);
}

int main()
{
    struct Employee emp1, emp2;

    printf("Enter Details of Employee 1\n");
    read(&emp1);

    printf("\nEnter Details of Employee 2\n");
    read(&emp2);

    printf("\nEmployee with Maximum Salary:\n");
    high_paid_employee(&emp1, &emp2);

    return 0;
}
```

**OUTPUT:**

```
[ita2536@mritlin harshan]$ cc ex1.2.c
[ita2536@mritlin harshan]$ vi ex1.2.c
[ita2536@mritlin harshan]$ ./a.out
Enter Details of Employee 1
Enter Name: harshan
Enter Employee ID: 1035
Enter Salary: 60000

Enter Details of Employee 2
Enter Name: raghav
Enter Employee ID: 1027
Enter Salary: 70000

Employee with Maximum Salary:

Employee Details
Name: raghav
Employee ID : 1027
Salary: 70000.00
[ita2536@mritlin harshan]$
```

**RESULT:**

Thus, the C program to read the details of two employees using pointers to structures and display the employee with the maximum salary using pass by address was implemented and executed successfully.